

□ Exercice 1 : Pointeurs

On considère le programme suivant :

```

1  int a = 4;
2  int b = 1;
3  int c = 2;
4  int* p;
5  int* q;
6  p = &a;
7  q = &c;
8  *p = *q + 1;
9  p = q;
10 q = &b;
11 *p = *p - *q;
12 *q = *q + 1;
13 *p *= *q;

```

Compléter le tableau suivant afin de donner l'état des variables au cours de l'exécution du programme :

	a	b	c	p	q
initialisation	4	1	2	?	?
p = &a;	4	1	2	&a	?
q = &c;					
*p = *q + 1;					
p = q;					
q = &b;					
*p = *p - *q;					
*q = *q + 1;					
*p *= *q;					

□ Exercice 2 : printf et scanf

1. Écrire l'instruction permettant d'afficher une variable `n` de type entier avec `printf`.
2. Écrire l'instruction permettant de saisir au clavier une variable `n` de type entier avec `scanf`.
3. Expliquer la différence entre le mode de passage de `n` dans ces deux fonctions.

□ Exercice 3 : Pointeurs

On considère le programme suivant :

```

1  #include <stdio.h>
2
3  int main()
4  {
5      int n = 7;
6      int *p;
7      *p = n;
8      printf("Valeur de n = %d \n", *p);
9      printf("Valeur de p= %p \n", p);
10 }

```

1. Ce programme est-il correct ?
2. Proposer une correction.

□ Exercice 4 : Incréments une variable

La fonction suivante doit incrémenter la variable `n` donnée en argument :

```

1  void incremente(int x) {
2      x = x + 1;
3  }

```

1. Commenter.

2. Proposer une correction.

❑ **Exercice 5** : *Fonction modifiant un paramètre*

Écrire en C une fonction `inverse` qui prend en argument un pointeur vers un booléen, ne renvoie rien et inverse la valeur de ce booléen (`true` devient `false` et inversement).

❑ **Exercice 6** : *Renvoyer un tableau*

```

1 // Renvoie un tableau contenant les entiers de 0 à n-1
2 int *cree_tab_entiers(int n)
3 {
4     int tab_entiers[n];
5     for (int i = 0; i < n; i++)
6     {
7         tab_entiers[i] = i;
8     }
9     return tab_entiers;
10 }
```

1. Lors de la compilation de la fonction ci-dessus, on obtient l'avertissement (*warning*) suivant : « *function returns address of local variable* ». Expliquer cet avertissement.
2. Dans quelle partie de la mémoire est stocké le tableau `tab_entiers` défini à la ligne 6 ?
3. Remplacer la ligne 6 par une allocation sur le tas.

❑ **Exercice 7** : *Deux plus grandes valeurs*

On souhaite écrire une fonction en C qui prend en argument un tableau d'entiers (de taille $n \geq 2$) et renvoie les deux plus grandes valeurs de ce tableau.

1. Proposer une solution utilisant un type structuré que l'on définira et donner alors la signature de la fonction.
2. Proposer une solution avec une fonction ne renvoyant rien mais modifiant deux paramètres passés par adresse. Donner la signature de la fonction dans ce cas.
3. Écrire les deux implémentations.

❑ **Exercice 8** : *Passer un pointeur*

On considère le programme suivant :

```

1 #include <stdio.h>
2
3 void init_pointer(int *p, int *adr)
4 {
5     //Initialise le pointeur p à l'adresse de val
6     p = adr;
7 }
8
9 int main()
10 {
11     int a = 42;
12     int *p = NULL;
13     init_pointer(p, &a);
14     printf("Valeur pointée par p : %d\n", *p);
15 }
```

Ce programme produit un *warning* à la compilation : *parameter 'p' set but not used* et une erreur de segmentation à l'exécution.

1. Expliquer ces deux résultats.
2. Proposer une correction afin que `init_pointer` soit conforme à sa spécification.

□ Exercice 9 : Matrice linéarisée

Pour allouer dynamiquement un tableau à deux dimensions (une matrice), la méthode privilégiée en C consiste à allouer un unique tableau à une dimension de taille $n \times m$ (où n est le nombre de lignes et m le nombre de colonnes).

1. Dans ce tableau linéarisé, quel indice correspond à l'élément situé à la ligne i et à la colonne j (en supposant que les indices de lignes et de colonnes commencent à 0) ?
2. Écrire une fonction `int* creer_identite(int n)` qui alloue dynamiquement sur le tas une matrice carrée de taille $n \times n$, l'initialise comme une matrice identité (des 1 sur la diagonale principale, des 0 partout ailleurs) et renvoie le pointeur vers cette matrice.
3. Écrire une fonction `void affiche_matrice(int* mat, int n)` permettant d'afficher cette matrice carrée dans le terminal.
4. Comment doit-on libérer la mémoire allouée par la fonction `creer_identite` depuis la fonction appelante ?

□ Exercice 10 : Fuite mémoire (Memory leak)

On considère le code C suivant :

```
1  #include <stdlib.h>
2
3  void simulations() {
4      int *donnees;
5      for (int i = 0; i < 1000; i++) {
6          donnees = malloc(10000 * sizeof(int));
7          // Traitement sur le tableau donnees (non détaillé ici)
8      }
9      free(donnees);
10 }
```

1. Ce programme compile sans erreur. Pourtant, il pose un grave problème de gestion de la mémoire. Lequel ?
2. Combien de blocs mémoires ont été alloués ? Combien ont été correctement libérés ?
3. Proposer une correction de la fonction `simulations` pour corriger cette erreur.